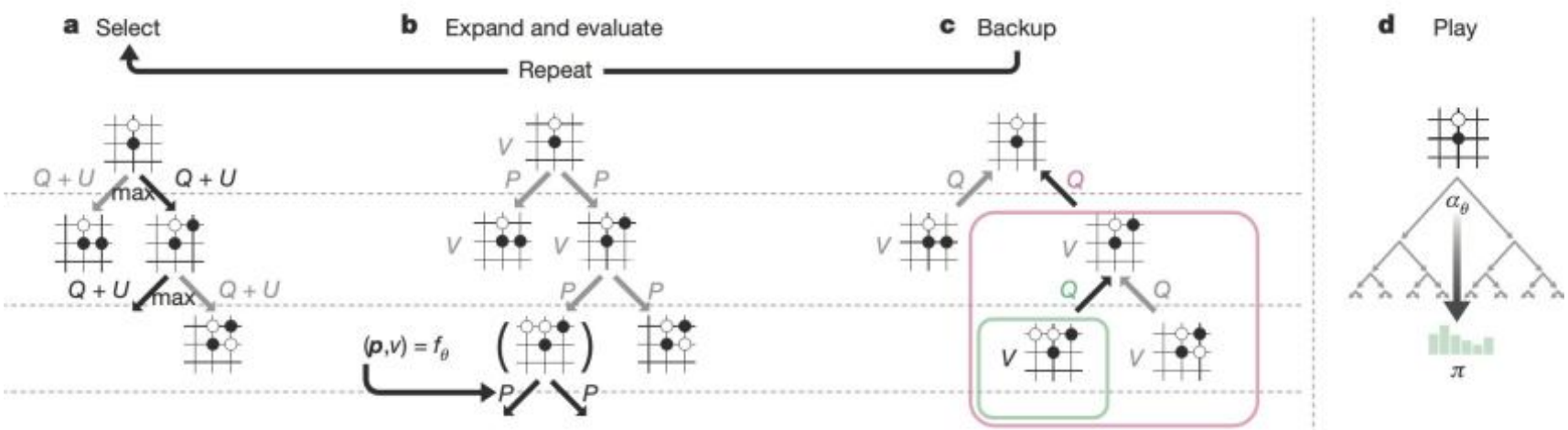
Reinforcement Learning

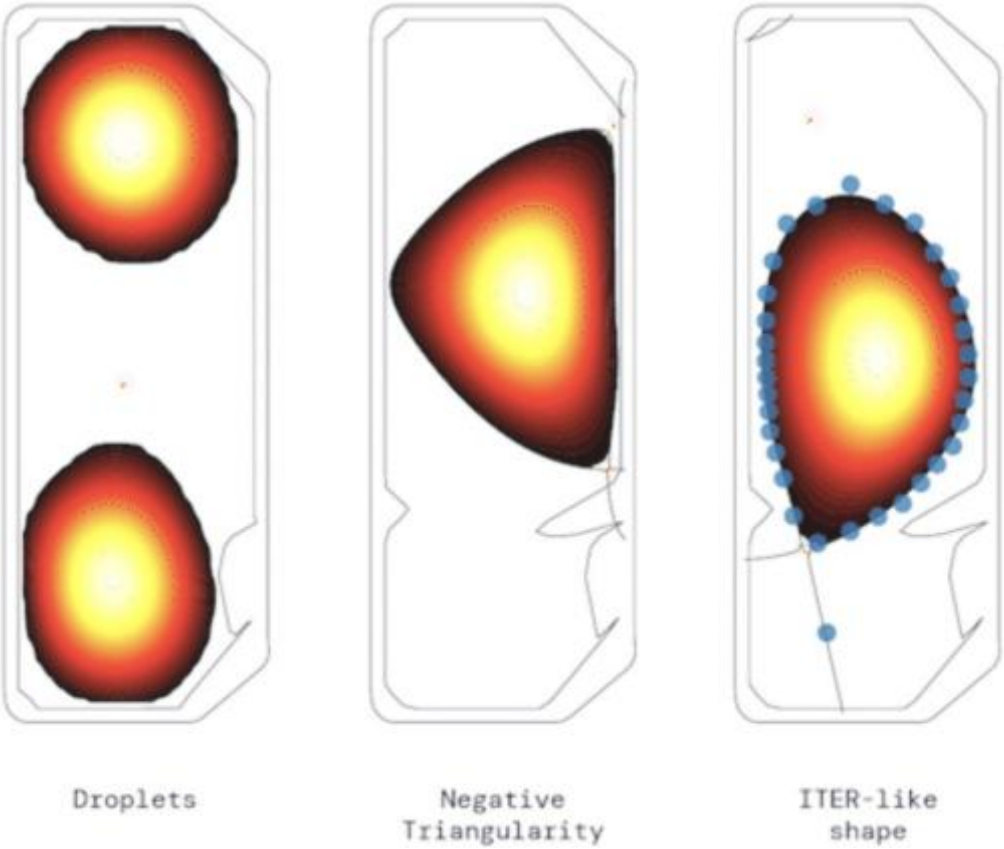
- Learning through experience/data to make good decisions under uncertainty
- Essential part of intelligence
- Builds strongly from theory and ideas starting in the 1950s with Richard Bellman
- A number of impressive successes in the last decade

Beyond Human Performance on the Board Game Go[1]



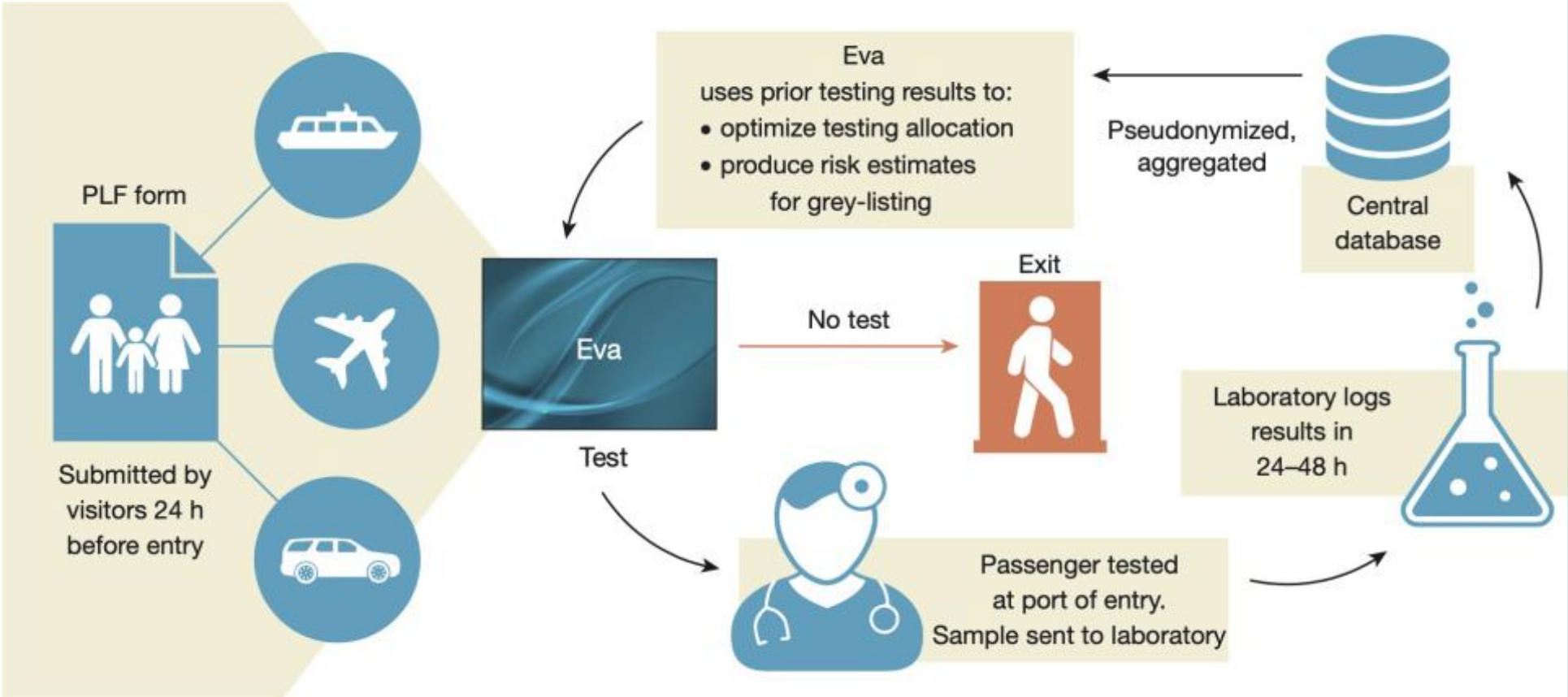
[1]Image credits: Silver et al. Nature 2017

Learning Plasma Control for Fusion Science[2]



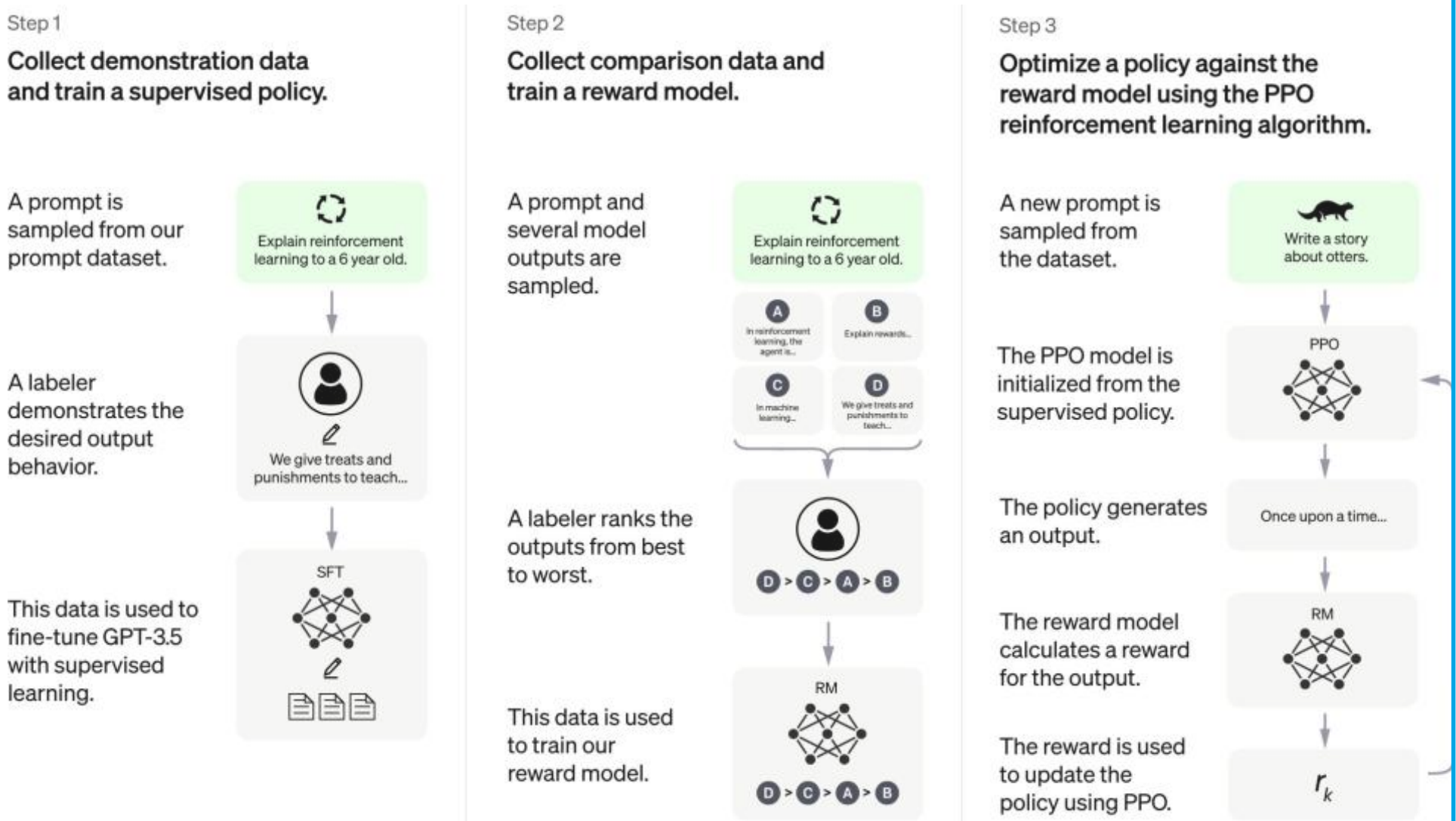
[2]Image credits: left Alain Herzog / EPFL, right DeepMind & SPC/EPFL. Degrave et

Efficient and targeted COVID-19 border testing via RL [3]

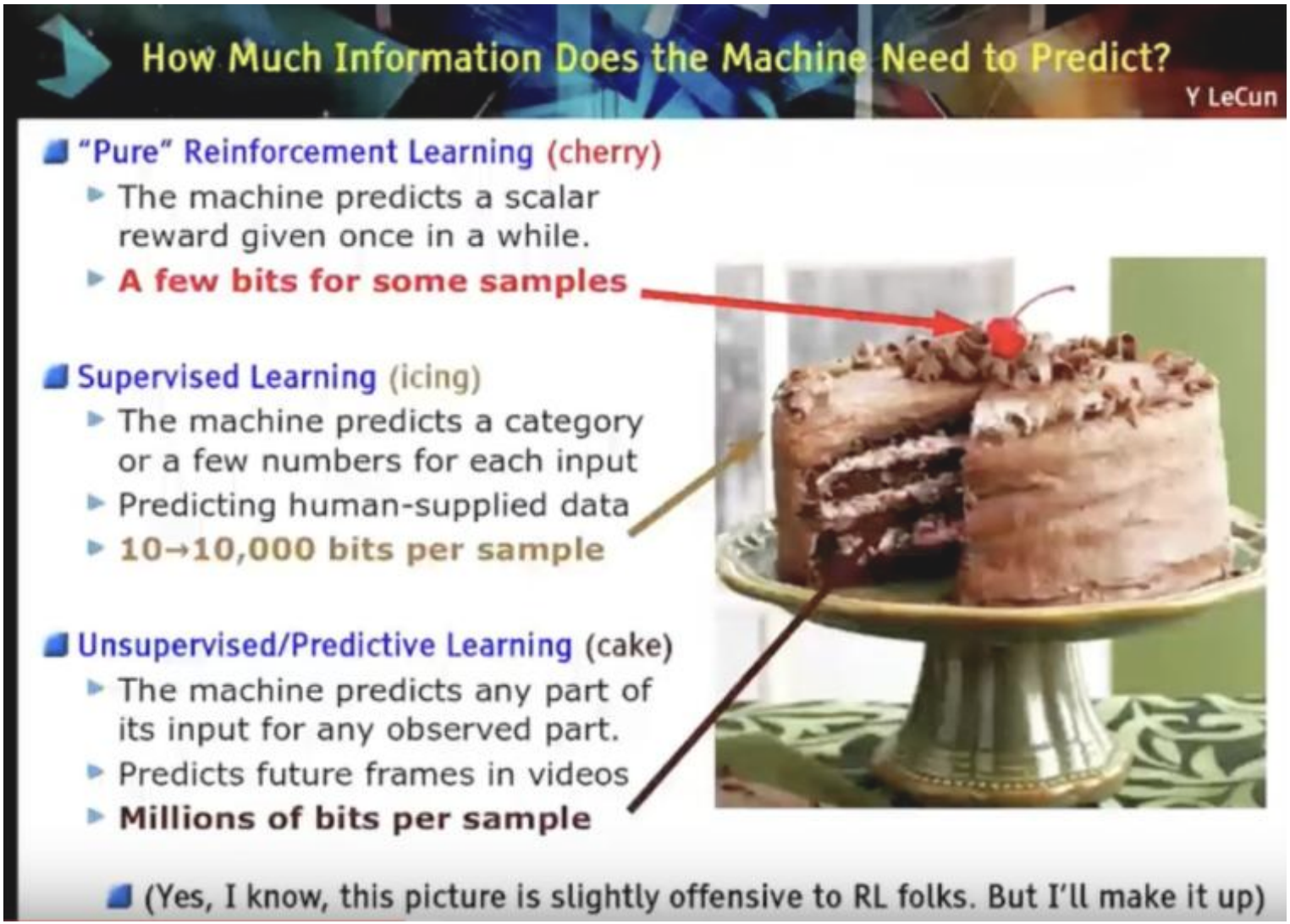


[3] Bastani et al. Nature 2021

ChatGPT (https://openai.com/blog/chatgpt/)



But Also Contentious[5]



Reinforcement Learning (Generally) Involves

- Optimization
- Delayed consequences
- Exploration
- Generalization

Optimization

- Goal is to find an optimal way to make decisions
  - Yielding best outcomes or at least very good outcomes
- Explicit notion of decision utility
- Example: finding minimum distance route between two cities given network of roads

Delayed Consequences

- Decisions now can impact things much later...
  - Saving for retirement
  - Finding a key in video game Montezuma's revenge
- Introduces two challenges
  - When planning: decisions involve reasoning about not just immediate benefit of a decision but also its longer term ramifications
  - When learning: temporal credit assignment is hard (what caused later high or low rewards?)

Exploration

- Learning about the world by making decisions
  - Agent as scientist
  - Learn to ride a bike by trying (and failing)
- Decisions impact what we learn about
  - Only get a reward for decision made
  - Don’t know what would have happened for other decision
  - If we choose to go to Stanford instead of MIT, we will have different later experiences...

Generalization

- Policy is mapping from past experience to action
- Why not just pre-program a policy?

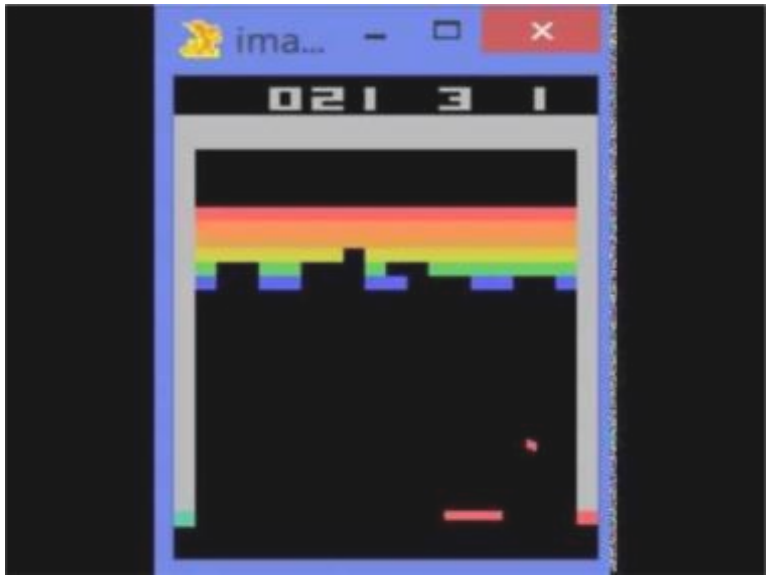


Figure: DeepMind Nature, 2015

RL vs Other AI and Machine Learning

	AI Planning	SL	UL	RL	IL
Optimization					
Learns from experience					
Generalization					
Delayed Consequences					
Exploration					

- SL = Supervised learning; UL = Unsupervised learning; RL = Reinforcement Learning; IL = Imitation Learning

RL vs Other AI and Machine Learning

	AI Planning	SL	UL	RL	IL
Optimization	X				
Learns from experience					
Generalization	X				
Delayed Consequences	X				
Exploration					

- SL = Supervised learning; UL = Unsupervised learning; RL = Reinforcement Learning; IL = Imitation Learning
- AI planning assumes have a model of how decisions impact environment

RL vs Other AI and Machine Learning

	AI Planning	SL	UL	RL	IL
Optimization	X				
Learns from experience		X			
Generalization	X	X			
Delayed Consequences	X				
Exploration					

- SL = Supervised learning; UL = Unsupervised learning; RL = Reinforcement Learning; IL = Imitation Learning
- Supervised learning has access to the correct labels

RL vs Other AI and Machine Learning

	AI Planning	SL	UL	RL	IL
Optimization	X				
Learns from experience		X	X		
Generalization	X	X	X		
Delayed Consequences	X				
Exploration					

- SL = Supervised learning; UL = Unsupervised learning; RL = Reinforcement Learning; IL = Imitation Learning
- Unsupervised learning has access to no labels

Imitation Learning

	AI Planning	SL	UL	RL	IL
Optimization	X			X	X
Learns from experience		X	X	X	X
Generalization	X	X	X	X	X
Delayed Consequences	X			X	X
Exploration				X	

- SL = Supervised learning; UL = Unsupervised learning; RL = Reinforcement Learning; IL = Imitation Learning
- Imitation learning typically assumes input demonstrations of good policies
- IL reduces RL to SL. **For many good reasons, IL is very popular.**

Two Problem Categories Where RL is Particularly Powerful

1. No examples of desired behavior: e.g. because the goal is to go beyond human performance or there is no existing data for a task.
2. Enormous search or optimization problem with delayed outcomes:

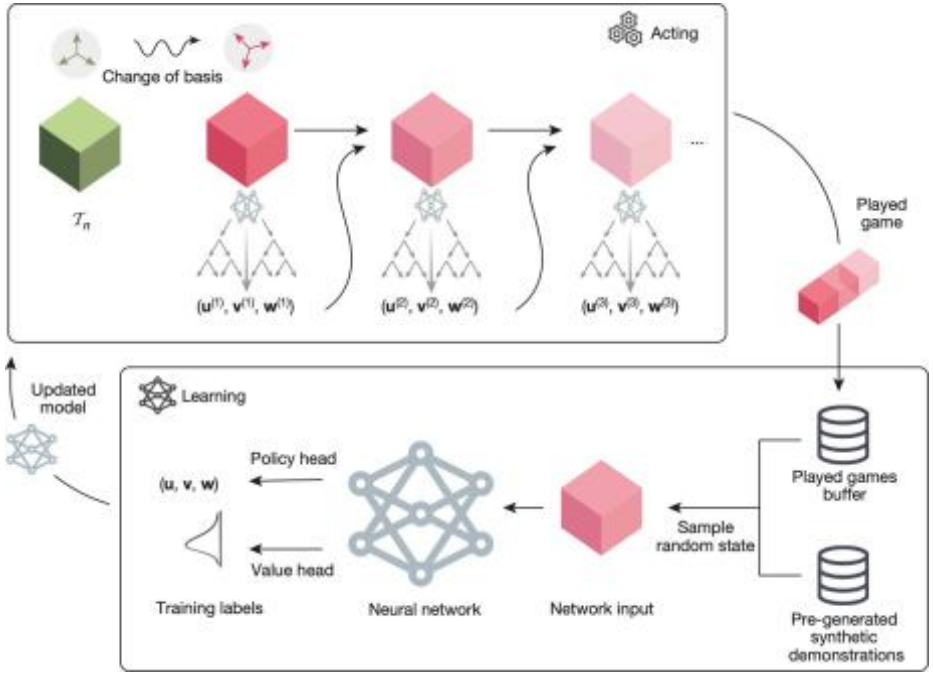


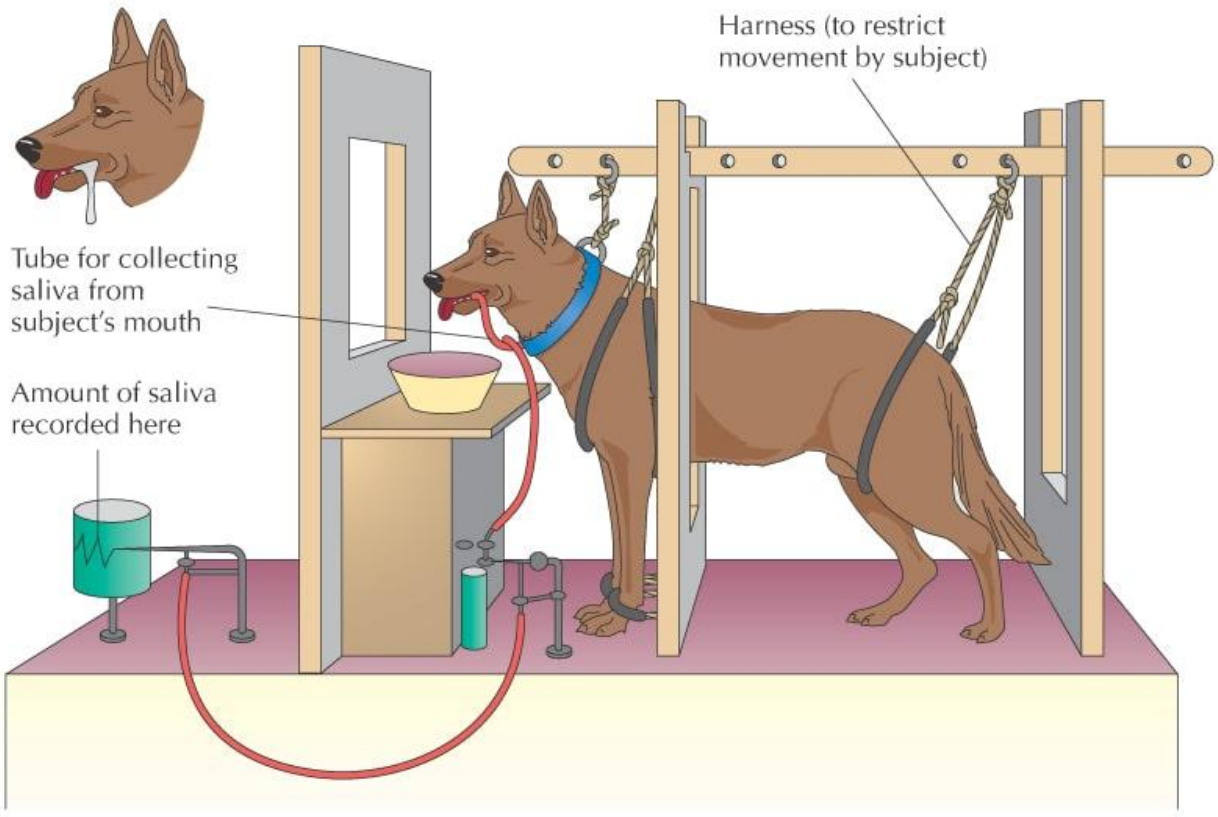
Figure: AlphaTensor. Fawzi et al. 2022

High Level Learning Goals[6]

- Define the key features of RL
- Given an application problem know how (and whether) to use RL for it
- Implement (in code) common RL algorithms
- Understand theoretical and empirical approaches for evaluating the quality of a RL algorithm

REINFORCEMENT LEARNING

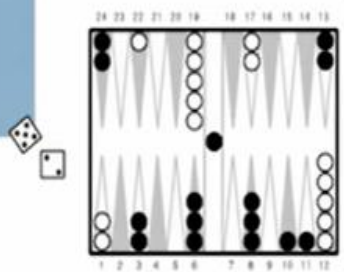
- A trial-and-error learning paradigm
- Learn about a system through interaction
- Inspired by behavioural psychology!
  - Pavlov's dog

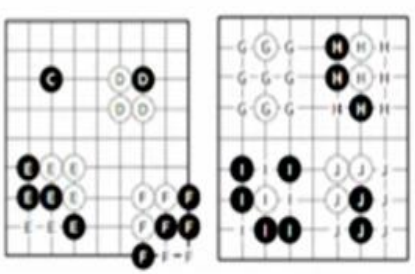

Harness (to restrict movement by subject)
Tube for collecting saliva from subject's mouth
Amount of saliva recorded here

REINFORCEMENT LEARNING

- Learning about stimuli and actions based on rewards and punishments alone.
- No detailed supervision available
- Trial-and-error learning
- Delayed rewards
- Sequence of actions required to obtain reward
- Associative learning required
  - Need to associate actions to states
- Learn about policies not just actions
- Typically in a stochastic world

RL Applications

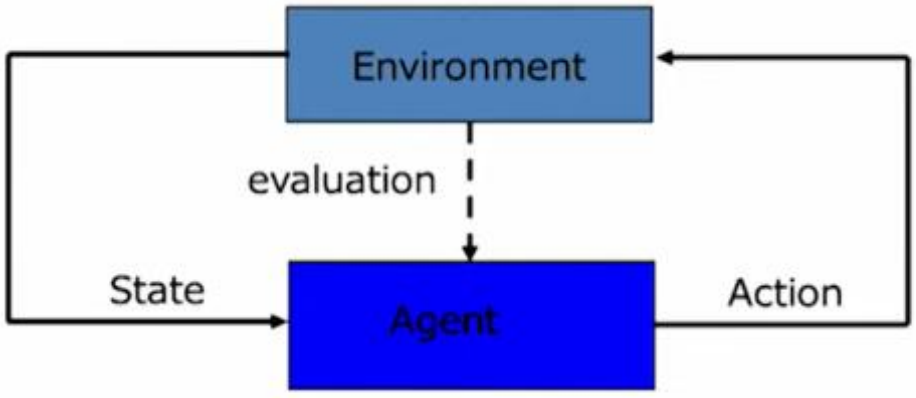




RL Framework

- Learn from close interaction
- Stochastic environment
- Noisy delayed scalar evaluation
- Maximize a measure of long term performance

RL Framework



Learn from close interaction

Stochastic environment

Noisy delayed scalar evaluation

Maximize a measure of long term performance

ELEMENTS OF RL

A policy defines the learning agent's way of behaving at a given time. Roughly speaking, a policy is a mapping from perceived states of the environment to actions to be taken when in those states.

A reward signal defines the goal in a reinforcement learning problem. On each time step, the environment sends to the reinforcement learning agent a single number, a reward. The agent's sole objective is to maximize the total reward it receives over the long run. The reward signal thus defines what are the good and bad events for the agent. In a biological system, we might think of rewards as analogous to the experiences of pleasure or pain

ELEMENTS OF RL

Whereas the reward signal indicates what is good in an immediate sense, a ***value function*** species what is good in the long run. Roughly speaking, the value of a state is the total amount of reward an agent can expect to accumulate over the future, starting from that state. Whereas rewards determine the immediate, intrinsic desirability of environmental states, values indicate the long-term desirability of states after taking into account the states that are likely to follow, and the rewards available in those states.

For example, a state might always yield a low immediate reward but still have a high value because it is regularly followed by other states that yield high rewards. Or the reverse could be true. To make a human analogy, rewards are somewhat like pleasure (if high) and pain (if low), whereas values correspond to a more refined and farsighted judgment of how pleased or displeased we are that our environment is in a particular state.

ELEMENTS OF RL

Rewards are in a sense primary, whereas values, as predictions of rewards, are secondary. Without rewards there could be no values, and the only purpose of estimating values is to achieve more reward. Nevertheless, it is values with which we are most concerned when making and evaluating decisions. Action choices are made based on value judgments.

ELEMENTS OF RL

model of the environment, This is something that mimics the behavior of the environment, or more generally, that allows inferences to be made about how the environment will behave.

For example, given a state and action, the model might predict the resultant next state and next reward. Models are used for planning, by which we mean any way of deciding on a course of action by considering possible future situations before they are actually experienced.

TEMPORAL DIFFERENCE

- Simple rule to explain complex behaviors
- Intuition: Prediction of outcome at time $t+1$ is better than the prediction at time t. Hence use the later prediction to adjust the earlier prediction.
- Has had profound impact in behavioral psychology and neuroscience!

”Learning is Not a Spectator Sport: Doing is Better than Watching for Learning from a MOOC”[8]

- In a psychology Massive Open Online Class, doing more activities seemed to yield a 6 times larger learning benefit compared to extra video watching or reading
- ”…it appears students actually spend substantially less time per activity (3.4 min) than reading a page (5.0 min)”
- *!* **Engaged practice is likely to be a more efficient and effective way to learn material.**
- To achieve the class learning goals, I encourage you to: do homework, attend problem sessions, do the check your understandings, and try past quiz or exam problems for practice without referring to solutions before you complete them

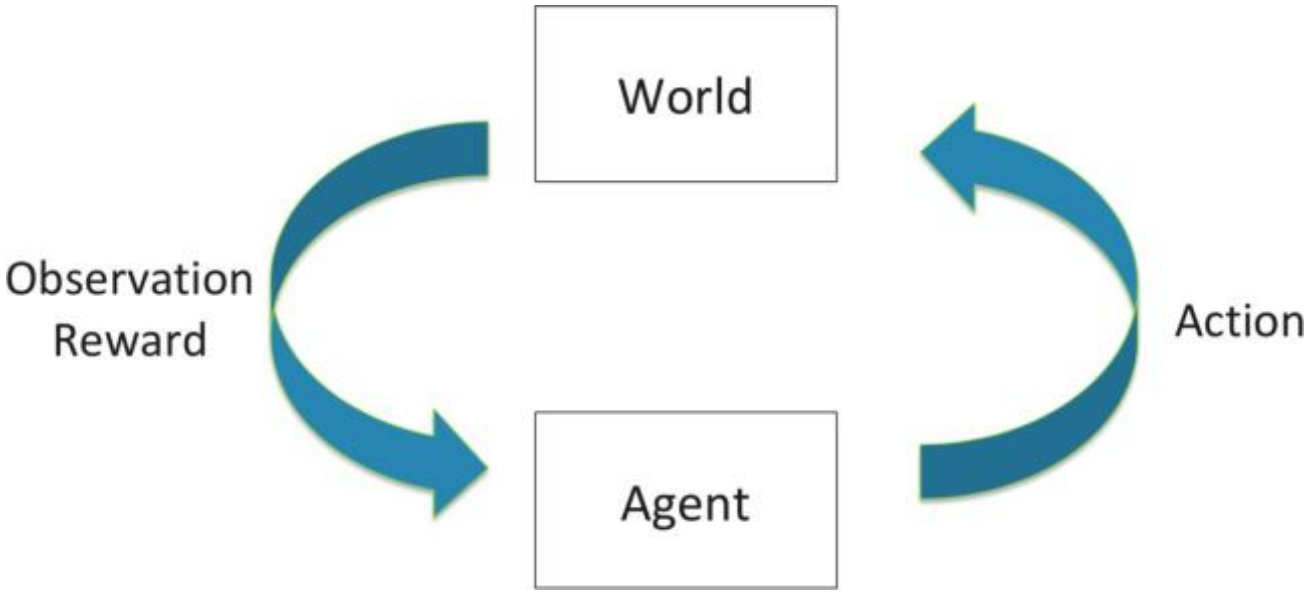
Refresher Exercise: AI Tutor as a Decision Process

- Student initially does not know either addition (easier) nor subtraction (harder)
- AI tutor agent can provide practice problems about addition or subtraction
- AI agent gets rewarded +1 if student gets problem right, -1 if get problem wrong
- Model this as a Decision Process. Define state space, action space, and reward model. What does the dynamics model represent? What would a policy to optimize the expected discounted sum of rewards yield?

Refresher Exercise: AI Tutor as a Decision Process

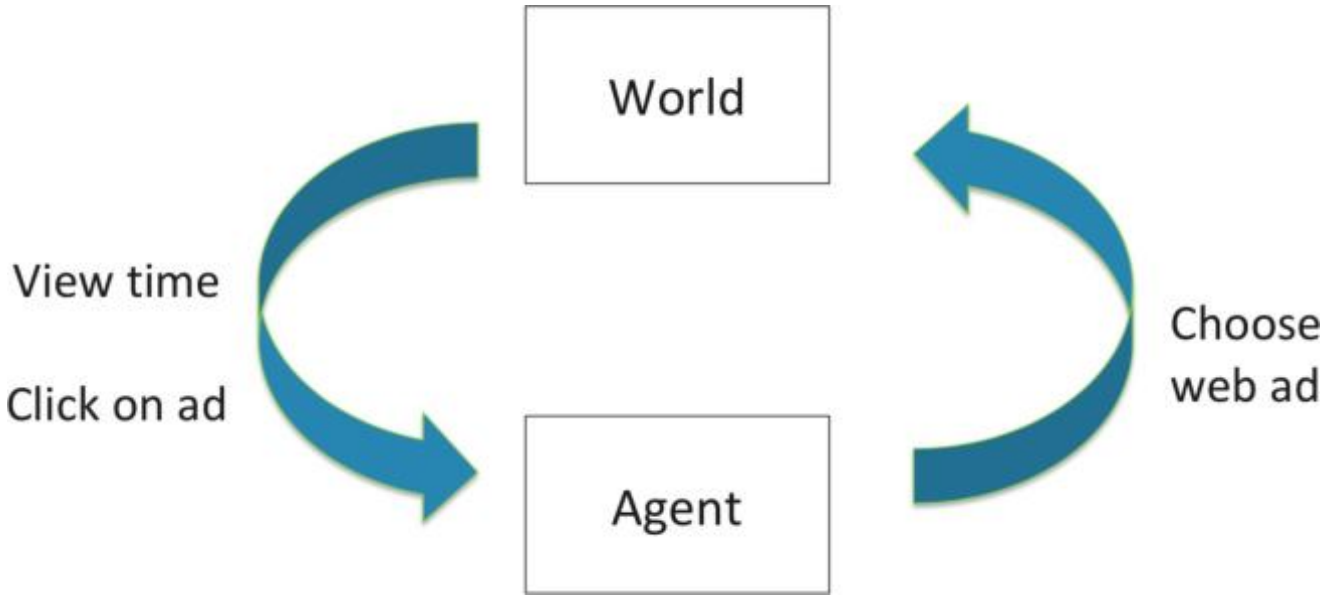
- State:
- Actions:
- Reward model:
- Meaning of dynamics model:

Sequential Decision Making



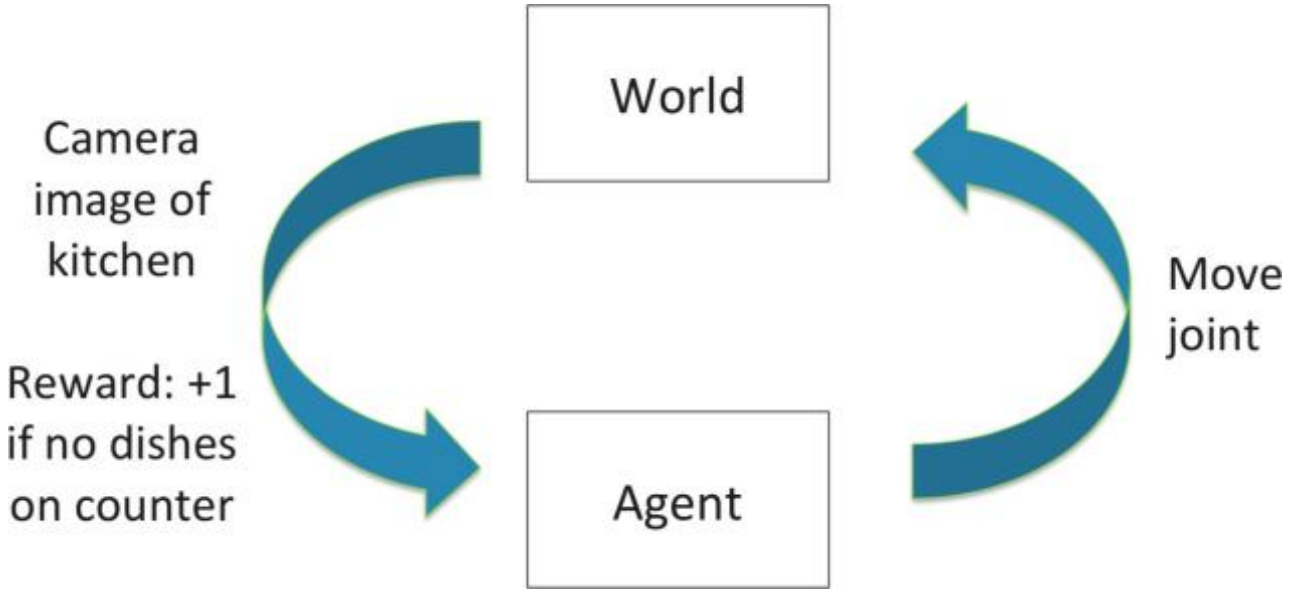
- Goal: Select actions to maximize total expected future reward
- May require balancing immediate & long term rewards

Example: Web Advertising



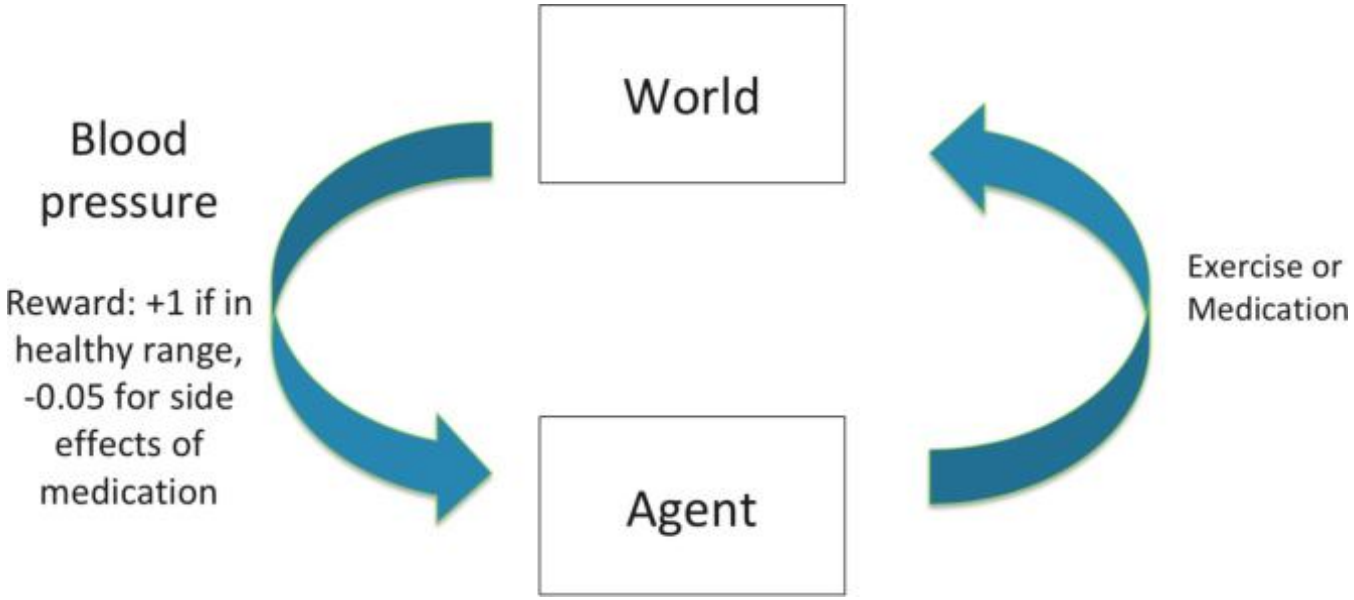
- Goal: Select actions to maximize total expected future reward
- May require balancing immediate & long term rewards

Example: Robot Unloading Dishwasher



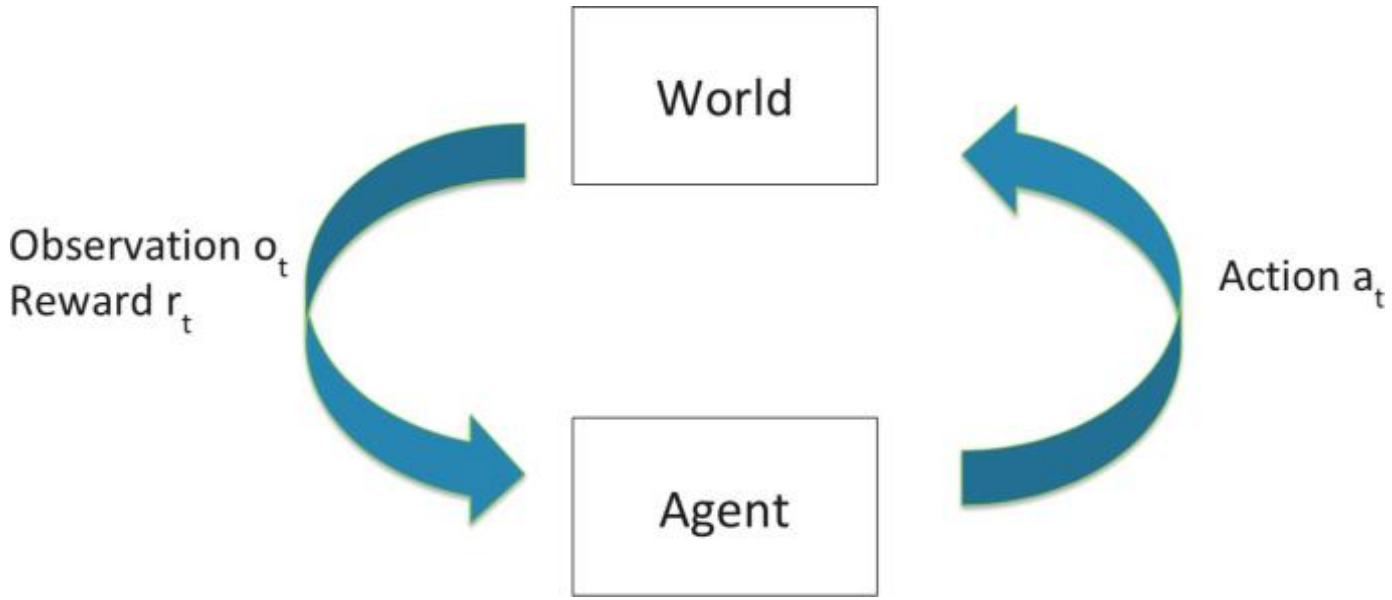
- Goal: Select actions to maximize total expected future reward
- May require balancing immediate & long term rewards

Example: Blood Pressure Control



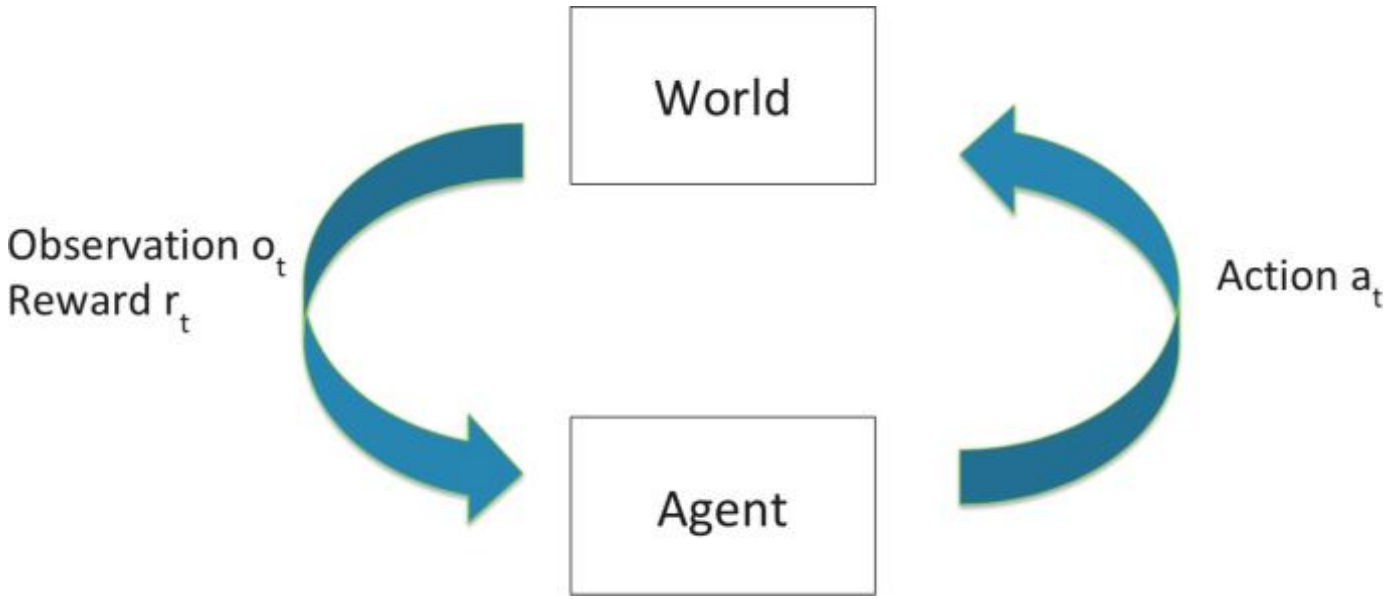
- Goal: Select actions to maximize total expected future reward
- May require balancing immediate & long term rewards

Sequential Decision Process: Agent & the World (Discrete Time)



- Each time step t:
  - Agent takes an action a_t
  - World updates given action a_t, emits observation o_t and reward r_t
  - Agent receives observation o_t and reward r_t

History: Sequence of Past Observations, Actions & Rewards



- History $h_t = (a_1, o_1, r_1, \ldots, a_t, o_t, r_t)$
- Agent chooses action based on history
- State is information assumed to determine what happens next
  - Function of history: $s_t = (h_t)$

Markov Assumption

- Information state: sufficient statistic of history
- State s_t is Markov if and only if:

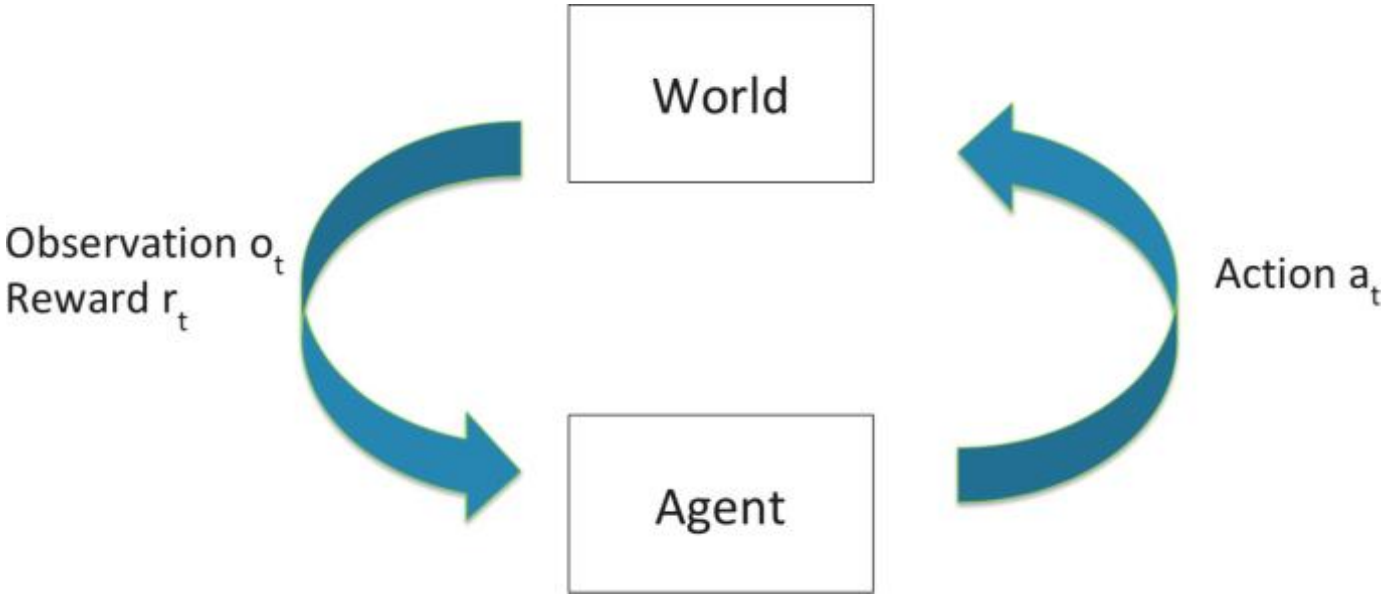
$$p(s_{t+1}|s_t, a_t) = p(s_{t+1}|h_t, a_t)$$

- Future is independent of past given present

Why is Markov Assumption Popular?

- Simple and often can be satisfied if include some history as part of the state
- In practice often assume most recent observation is sufficient statistic of history: $s_t = o_t$
- State representation has big implications for:
  - Computational complexity
  - Data required
  - Resulting performance

Types of Sequential Decision Processes



- Is state Markov? Is world partially observable? (POMDP)
- Are dynamics deterministic or stochastic?
- Do actions influence only immediate reward (bandits) or reward and next state ?

Example: Mars Rover as a Markov Decision Process

s_1	s_2	s_3	s_4	s_5	s_6	s_7

Figure: Mars rover image: NASA/JPL-Caltech

- States: Location of rover ($s_1, \ldots, s_7$)
- Actions: TryLeft or TryRight
- Rewards:
  - +1 in state s_1
  - +10 in state s_7
  - 0 in all other states

MDP Model

- Agent's representation of how world changes given agent's action
- Transition / dynamics model predicts next agent state

$$p(s_{t+1} = s' | s_t = s, a_t = a)$$

- Reward model predicts immediate reward

$$r(s_t = s, a_t = a) = \mathrm{E}[r_t | s_t = s, a_t = a]$$

Example: Mars Rover Stochastic Markov Model

s_1	s_2	s_3	s_4	s_5	s_6	s_7
$\hat{r} = 0$	$\hat{r} = 0$	$\hat{r} = 0$	$\hat{r} = 0$	$\hat{r} = 0$	$\hat{r} = 0$	$\hat{r} = 0$

- Numbers above show RL agent's reward model
- Part of agent's transition model:
  - $0.5 = P(s_1|s_1, \text{TryRight}) = P(s_2|s_1, \text{TryRight})$
  - $0.5 = P(s_2|s_2, \text{TryRight}) = P(s_3|s_2, \text{TryRight}) \cdots$
- Model may be wrong

Policy

- Policy π determines how the agent chooses actions
- $\pi : S \rightarrow A$, mapping from states to actions
- Deterministic policy:

$$\pi(s) = a$$

- Stochastic policy:

$$\pi(a|s) = Pr(a_t = a|s_t = s)$$

Example: Mars Rover Policy

s_1	s_2	s_3	s_4	s_5	s_6	s_7

- $\uparrow(s_1) = \uparrow(s_2) = \cdots = \uparrow(s_7) =$ TryRight
- Quick check your understanding: is this a deterministic policy or a stochastic policy?

Evaluation and Control

- Evaluation
  - Estimate/predict the expected rewards from following a given policy
- Control
  - Optimization: find the best policy

MULTI-ARMS BANDITS

The most important feature distinguishing reinforcement learning from other types of learning is that it uses training information that evaluates the actions taken rather than instructs by giving correct actions

Purely evaluative feedback indicates how good the action taken is, but not whether it is the best or the worst action possible

Evaluative feedback is the basis of methods for function optimization, including evolutionary methods.

MULTI–ARMS BANDITS

This nonassociative setting is the one in which most prior work involving evaluative feedback has been done, and it avoids much of the complexity of the full reinforcement learning problem

The particular nonassociative, evaluative feedback problem that we explore is a simple version of the n-armed bandit problem.

MULTI-ARMS BANDITS

You are faced repeatedly with a choice among n different options, or actions. After each choice you receive a numerical reward chosen from a stationary probability distribution that depends on the action you selected. Your objective is to maximize the expected total reward over some time period, for example, over 1000 action selections,or time steps.

This is the original form of the n-armed bandit problem, so named by analogy to a slot machine, or \one-armed bandit," except that it has n levers instead of one. Through repeated action selections you are to maximize your winnings by concentrating your actions on the best levers.

MULTI–ARMS BANDITS

Another analogy is that of a doctor choosing between experimental treatments for a series of seriously ill patients.

Each action selection is a treatment selection, and each reward is the survival or well-being of the patient.

MULTI-ARMS BANDITS

In our n-armed bandit problem, each action has an expected or mean reward given that the action is selected; let us call this the value of that action.

If you knew the value of each action, then it would be trivial to solve the n-armed bandit problem: you would always select the action with highest value.

We assume that you do not know the action values with certainty, although you may have estimates.

EXPLORATION AND EXPLOITATION

If you maintain estimates of the action values, then at any time step there is at least one action whose estimated value is greatest.

We call this a greedy action. If you select a greedy action, we say that you are exploiting your current knowledge of the values of the actions.

If instead you select one of the nongreedy actions, then we say you are exploring, because this enables you to improve your estimate of the nongreedy action's value.

EXPLORATION AND EXPLOITATION

Exploitation is the right thing to do to maximize the expected reward on the one step, but exploration may produce the greater total reward in the long run.

For example, suppose the greedy action's value is known with certainty, while several other actions are estimated to be nearly as good but with substantial uncertainty.

The uncertainty is such that at least one of these other actions probably is actually better than the greedy action, but you don't know which one.

EXPLORATION AND EXPLOITATION

If you have many time steps ahead on which to make action selections, then it may be better to explore the nongreedy actions and discover which of them are better than the greedy action.

Reward is lower in the short run, during exploration, but higher in the long run because after you have discovered the better actions, you can exploit them many times.

Because it is not possible both to explore and to exploit with any single action selection, one often refers to the conflict" between exploration and exploitation.

EXPLORATION AND EXPLOITATION

There are many sophisticated methods for balancing exploration and exploitation for particular mathematical formulations of the n-armed bandit and related problems.

However, most of these methods make strong assumptions about stationarity and prior knowledge

ACTION–VALUE METHODS

true (actual) value of action a as q(a), and the estimated value on the tth time step as Qt(a).
Recall that the true value of an action is the mean reward received when that action is selected.
One natural way to estimate this is by averaging the rewards actually received when the action was selected.

In other words, if by the ith time step action a has been Chosen Nt(a) times prior to t, yielding rewards R1;R2;.....;RNt(a), then its value is estimated to be

$$Q_t(a) = \frac{R_1 + R_2 + \cdots + R_{N_t(a)}}{N_t(a)}.$$

GREEDY METHOD

The simplest action selection rule is to select the action (or one of the actions) with highest estimated action value, that is, to select at step t one of the greedy actions, At*, for which
Qt(At*) = maxa Qt(a).

This greedy action selection method can be written as

$$A_t \doteq \arg\max_a Q_t(a),$$

GREEDY METHOD

where argmaxa denotes the value of a at which the expression that follows is maximized (with ties broken arbitrarily).

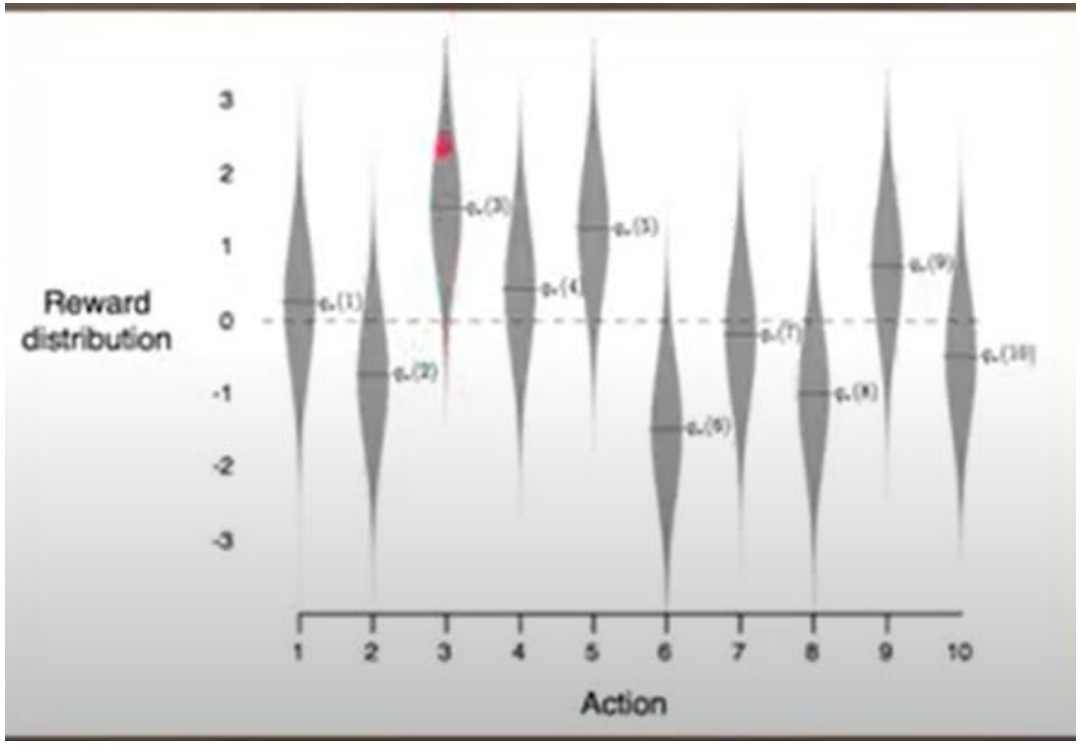
Greedy action selection always exploits current knowledge to maximize immediate reward; it spends no time at all sampling apparently inferior actions to see if they might really be better.

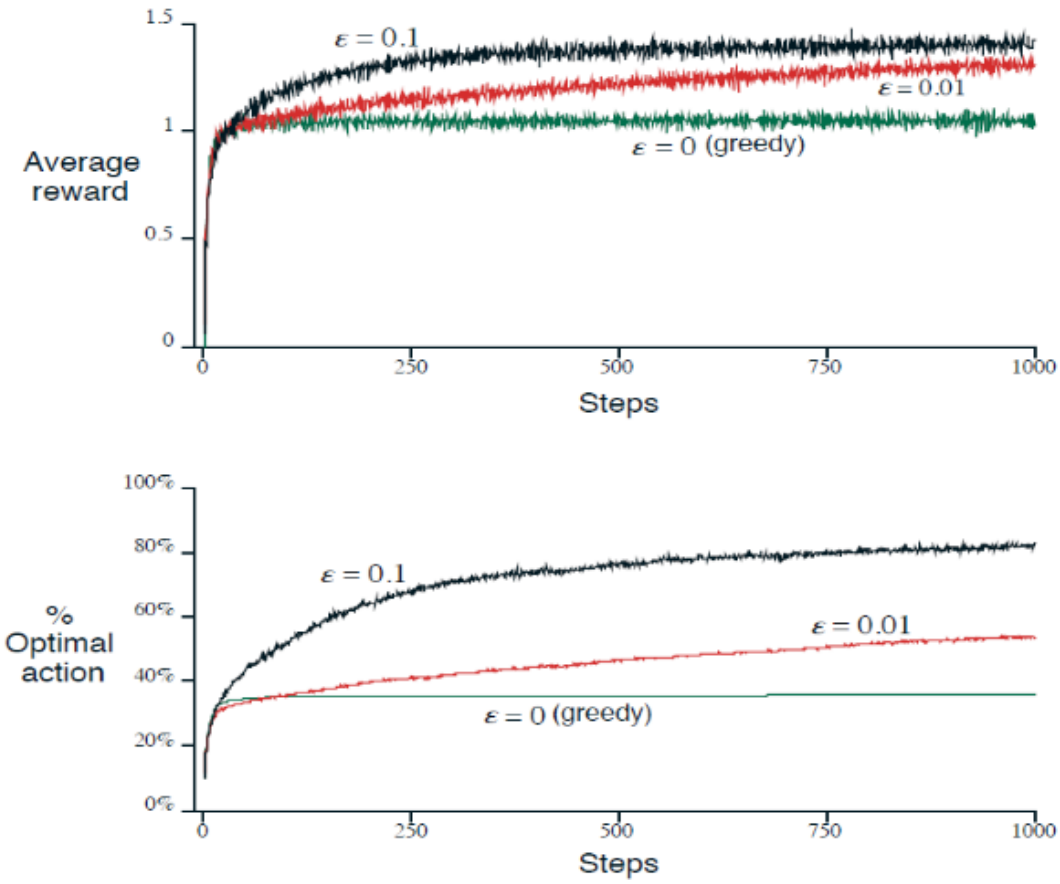
A simple alternative is to behave greedily most of the time, but every once in a while, say with small probability instead to select randomly from amongst all the actions with equal probability independently of the action value estimates. We call methods using this near-greedy action selection rule epsilon-greedy methods

EXAMPLE

To roughly assess the relative effectiveness of the greedy and epsilon greedy methods, we compared them numerically on a suite of test problems.

This was a set of 2000 randomly generated n-armed bandit tasks with n = 10. For each bandit, the action values, q(a), a = 1; : : : ; 10, were selected according to a normal (Gaussian) distribution with mean 0 and variance 1. On tth time step with a given bandit, the actual reward Rt was the q(At) for the bandit (where
At was the action selected) plus a normally distributed noise term that was mean 0 and variance 1. Averaging over bandits, we can plot the performance and behavior of various methods as they improve with experience over 1000 steps,
We call this suite of test tasks the 10-armed testbed.


Reward distribution
3
2
1
0
-1
-2
-3
1
2
3
4
5
6
7
8
9
10
Action


1.5
1
0.5
0
Average reward
ε = 0.1
ε = 0.01
ε = 0 (greedy)
0
250
500
750
1000
Steps
100%
80%
60%
40%
20%
0%
% Optimal action
ε = 0.1
ε = 0.01
ε = 0 (greedy)
0
250
500
750
1000
Steps

ISSUES

A problem with this straightforward implementation
is that its memory and computational requirements grow over time
without bound.

That is, each additional reward following a selection of action a
requires more memory to store it and results in more computation
being required to determine Qt(a).

EFFICIENT SAMPLE AVERAGING

Constant memory and constant computation.

$$NewEstimate \leftarrow OldEstimate + StepSize\left[Target - OldEstimate\right].$$

$$\begin{aligned}
Q_{n+1} &= \frac{1}{n}\sum_{i=1}^{n} R_i \\
&= \frac{1}{n}\left(R_n + \sum_{i=1}^{n-1} R_i\right) \\
&= \frac{1}{n}\left(R_n + (n-1)\frac{1}{n-1}\sum_{i=1}^{n-1} R_i\right) \\
&= \frac{1}{n}\left(R_n + (n-1)Q_n\right) \\
&= \frac{1}{n}\left(R_n + nQ_n - Q_n\right) \\
&= Q_n + \frac{1}{n}\left[R_n - Q_n\right],
\end{aligned}$$

BANDIT ALGORITHM

A simple bandit algorithm

Initialize, for $a = 1$ to k:

$Q(a) \leftarrow 0$

$N(a) \leftarrow 0$

Loop forever:

$$A \leftarrow \begin{cases} \arg\max_a Q(a) & \text{with probability } 1-\varepsilon \quad \text{(breaking ties randomly)} \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$$

$R \leftarrow bandit(A)$

$N(A) \leftarrow N(A) + 1$

$Q(A) \leftarrow Q(A) + \frac{1}{N(A)}\left[R - Q(A)\right]$

Adjusting Step-Size for Non-Stationary Rewards

- α in (0, 1] is constant
- $\alpha = 1, R_n = 10, Q_n = 5 \rightarrow Q_{n+1} = 10$
- $\alpha = 0.5, R_n = 10, Q_n = 5 \rightarrow Q_{n+1} = 7.5$

$$Q_{n+1} \doteq Q_n + \alpha\left[R_n - Q_n\right],$$

EXPONENTIAL RECENCY WEIGHTED AVERAGE

$$
\begin{aligned}
Q_{n+1} &= Q_n + \alpha\left[R_n - Q_n\right] \\
&= \alpha R_n + (1-\alpha)Q_n \\
&= \alpha R_n + (1-\alpha)\left[\alpha R_{n-1} + (1-\alpha)Q_{n-1}\right] \\
&= \alpha R_n + (1-\alpha)\alpha R_{n-1} + (1-\alpha)^2 Q_{n-1} \\
&= \alpha R_n + (1-\alpha)\alpha R_{n-1} + (1-\alpha)^2 \alpha R_{n-2} + \\
&\qquad \cdots + (1-\alpha)^{n-1}\alpha R_1 + (1-\alpha)^n Q_1 \\
&= (1-\alpha)^n Q_1 + \sum_{i=1}^{n} \alpha(1-\alpha)^{n-i} R_i.
\end{aligned}
$$

Feature	Multi-Armed Bandit (MAB)	Reinforcement Learning (RL)
Environment	No states—just actions and rewards	Has **states**, **actions**, **rewards**, and **transitions**
Memory of past actions	Independent trials—no long-term consequences	Actions affect future states and rewards
Exploration vs. Exploitation	Yes, key idea	Yes, but more complex due to state transitions
Goal	Maximize total reward over trials	Learn a policy (mapping from states to actions) to maximize cumulative reward over time
Complexity	Simpler	More complex due to dynamics of the environment

Making Sequences of Good Decisions Given a Model of the World

- Assume finite set of states and actions
- Given models of the world (dynamics and reward)
- Evaluate the performance of a particular decision policy
- Compute the best policy
- This can be viewed as an AI planning problem

Making Sequences of Good Decisions Given a Model of the World

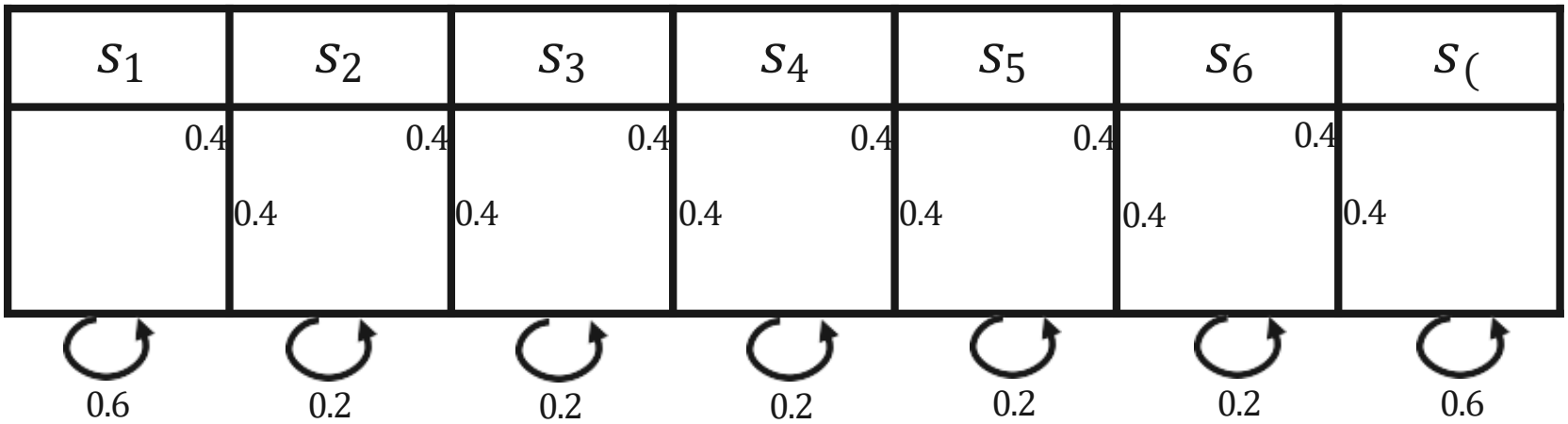
- Markov Processes
- Markov Reward Processes (MRPs)
- Markov Decision Processes (MDPs)
- Evaluation and Control in MDPs

Markov Process or Markov Chain

- Memoryless random process
  - Sequence of random states with Markov property
- Definition of Markov Process
  - S is a (finite) set of states (s 2 S)
  - P is dynamics/transition model that specifices $p(s_{t+1} = s^0 | s_t = s)$
- Note: no rewards, no actions
- If finite number (N) of states, can express P as a matrix

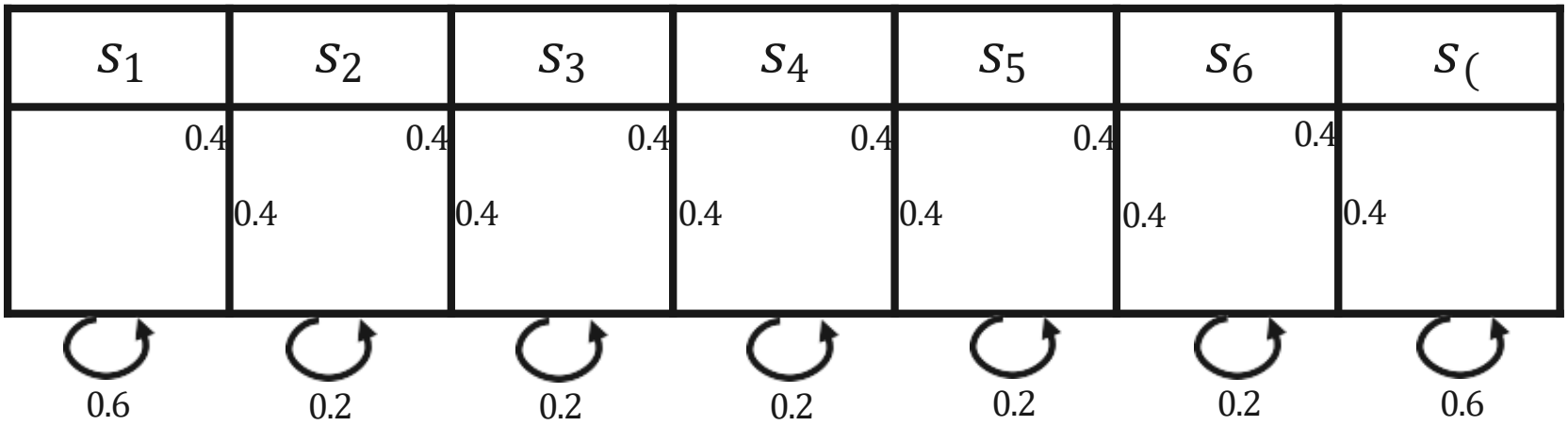
$$P = \begin{pmatrix} P(s_1|s_1) & P(s_2|s_1) & \cdots & P(s_N|s_1) \\ P(s_1|s_2) & P(s_2|s_2) & \cdots & P(s_N|s_2) \\ \vdots & \vdots & \ddots & \vdots \\ P(s_1|s_N) & P(s_2|s_N) & \cdots & P(s_N|s_N) \end{pmatrix}$$

Example: Mars Rover Markov Chain Transition Matrix, P



$$P = \begin{pmatrix} 0.6 & 0.4 & 0 & 0 & 0 & 0 & 0 \\ 0.4 & 0.2 & 0.4 & 0 & 0 & 0 & 0 \\ 0 & 0.4 & 0.2 & 0.4 & 0 & 0 & 0 \\ 0 & 0 & 0.4 & 0.2 & 0.4 & 0 & 0 \\ 0 & 0 & 0 & 0.4 & 0.2 & 0.4 & 0 \\ 0 & 0 & 0 & 0 & 0.4 & 0.2 & 0.4 \\ 0 & 0 & 0 & 0 & 0 & 0.4 & 0.6 \end{pmatrix}$$

Example: Mars Rover Markov Chain Episodes



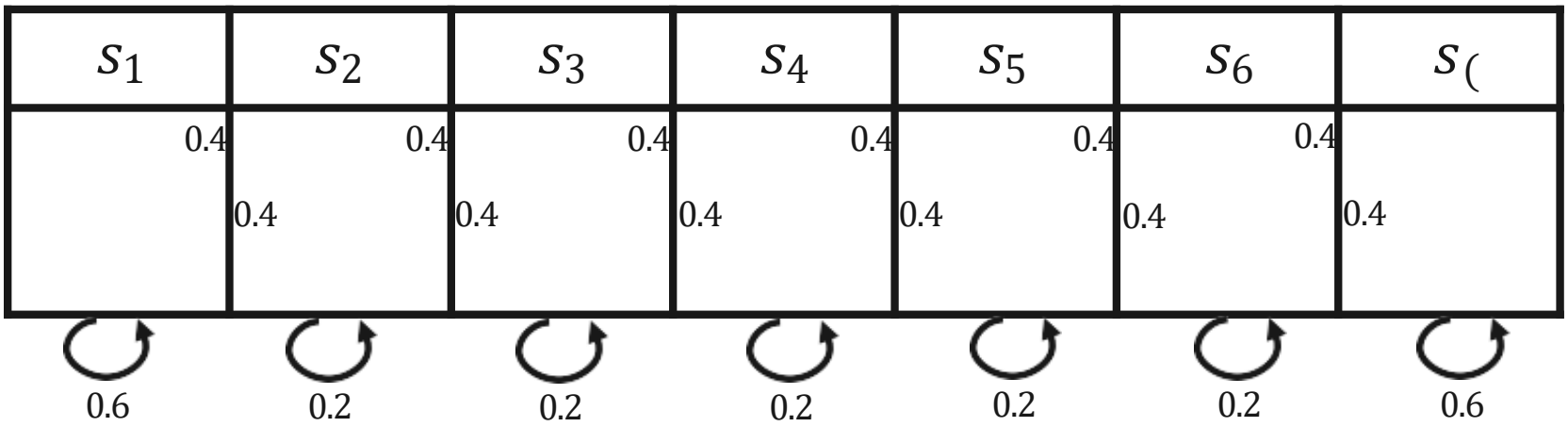
Example: Sample episodes starting from S4

- $s_4, s_5, s_6, s_7, s_7, s_7, \ldots$
- $s_4, s_4, s_5, s_4, s_5, s_6, \ldots$
- $s_4, s_3, s_2, s_1, \ldots$

Markov Reward Process (MRP)

- Markov Reward Process is a Markov Chain + rewards
- Definition of Markov Reward Process (MRP)
  - S is a (finite) set of states (s 2 S)
  - P is dynamics/transition model that specifices $P(s_{t+1} = s^0 | s_t = s)$
  - R is a reward function $R(s_t = s) = \mathrm{E}[r_t | s_t = s]$
  - Discount factor ц 2 $[0, 1]$
- Note: no actions
- If finite number (N) of states, can express R as a vector

Example: Mars Rover Markov Reward Process



- Reward: +1 in s_1, +10 in s_7, 0 in all other states

Return & Value Function

- Definition of Horizon (H)
  - Number of time steps in each episode
  - Can be infinite
  - Otherwise called **finite** Markov reward process
- Definition of Return, G_t (for a Markov Reward Process)
  - Discounted sum of rewards from time step t to horizon H

$$G_t = r_t + \text{ц} r_{t+1} + \text{ц}^2 r_{t+2} + \cdots + \text{ц}^{H-1} r_{t+H-1}$$

- Definition of State Value Function, $V(s)$ (for a Markov Reward Process)
  - Expected return from starting in state s

$$V(s) = \mathrm{E}[G_t | s_t = s] = \mathrm{E}[r_t + \text{ц} r_{t+1} + \text{ц}^2 r_{t+2} + \cdots + \text{ц}^{H-1} r_{t+H-1} | s_t = s]$$

Discount Factor

- Mathematically convenient (avoid infinite returns and values)
- Humans often act as if there's a discount factor < 1
- If episode lengths are always finite ($H < 1$), can use $\gamma = 1$

Discount Factor

- Mathematically convenient (avoid infinite returns and values)
- Humans often act as if there's a discount factor < 1
- $\gamma = 0$: Only care about immediate reward
- $\gamma = 1$: Future reward is as beneficial as immediate reward
- If episode lengths are always finite ($H < 1$), can use $\gamma = 1$